RepairDesk — Device-Repair Chat & Ticket-Intake Assistant

> **Short description:** An AI front desk for a device-repair shop, in Python with the OpenAI API. It answers general questions in a streaming chat, and turns a free-text problem ("my phone screen is cracked") into a clean, fixed-shape repair ticket. A router reads each message and sends it to the right agent automatically.

Tech: Python · OpenAI API (raw SDK) · Pydantic · **Type:** Multi-agent (Concierge + Intake, with a router) · **Level:** Beginner-friendly

Overview

RepairDesk is the front desk of a repair shop, done by AI. You type a message; it decides what you need and acts:

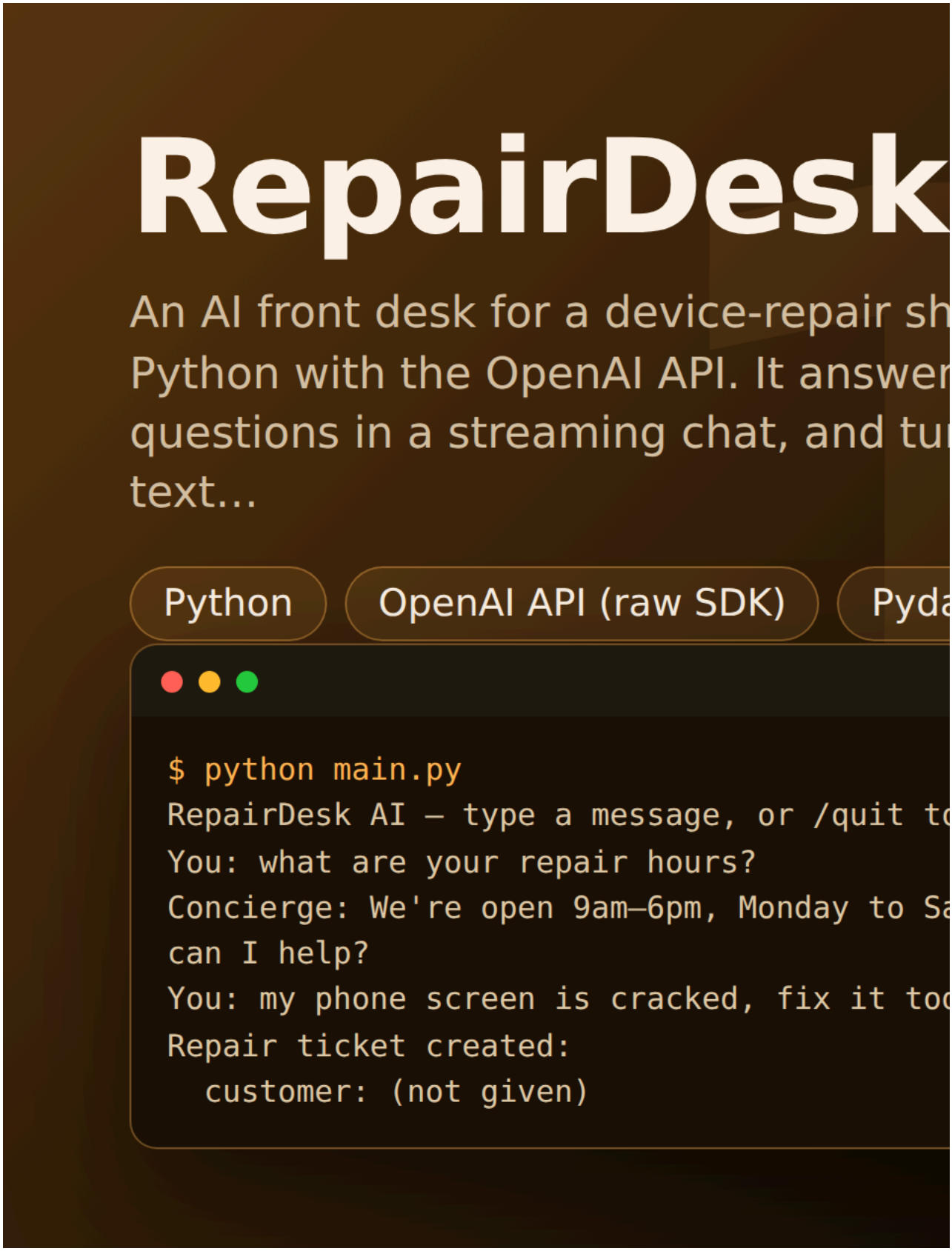
- **General questions** (hours, prices, services) → the **Concierge** answers in a live, streaming chat that remembers the conversation.
- **A repair problem** → the **Intake** agent turns the free text into a checked **repair ticket**: customer name (only if given), device, urgency, and a short summary.
- A **Router** reads every message first and picks the right agent — no menus.

The point of the project is the strict shape: the ticket can only hold real device types and real urgency levels, and the name is left empty rather than invented. That keeps bad data out of your system.

Features

- **Two agents + a router** — the router sends each message to the Concierge or the Intake agent, with a logged reason.
- **Structured tickets** — a Pydantic `RepairTicket`; the model must answer in exactly that shape, or the agent returns nothing.
- **Never invents a name** — if the customer didn't say their name, the ticket's name stays empty.
- **Real chat** — remembers earlier turns, and streams the reply word by word.
- **Stays under the limit** — trims old turns so a long chat never overflows the model's context.
- **Safe errors** — a missing key stops at startup; a bad key, a rate limit, or a too-long message each get one clear line, no crash.

Demo




```
$ python main.py
RepairDesk AI — type a message, or /quit to exit.
You: what are your repair hours?
Concierge: We're open 9am–6pm, Monday to Saturday. How can I help?
You: my phone screen is cracked, fix it today
```

```
Repair ticket created:
  customer: (not given)
  device:   phone
  urgency:  high
  summary:  my phone screen is cracked, fix it today
You: /quit
```

Architecture

```
your message
   │
   ▼
Router (decide)  ──►  RouteDecision {agent, reason}
   │
   ├── "concierge" ──►  Concierge agent  ──►  streamed answer (with memory)
   └── "intake"    ──►  Intake agent     ──►  RepairTicket (checked shape)
```

- **One decision first** — the router runs before any agent; it falls back to the Concierge if unsure.
- **Fixed shapes** — `RepairTicket` and `RouteDecision` are Pydantic models, so a wrong value is caught, not stored.
- **One file talks to OpenAI** — `src/models/llm.py`; everything else uses it.

Project Structure

```
<repo folder>/   (RepairDesk-Device-Repair-Chat-And-Ticket-Intake-Assistant/)
├── data/
│   └── sample/
│       └── messages.json       sample problems + routing cases
├── scripts/
│   └── intake_samples.py       runs Intake on every sample message
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── concierge.py        concierge_agent() -> str
│   │   ├── intake.py           intake_agent() -> RepairTicket | None
│   │   └── router.py           decide() -> RouteDecision
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              create_client(), send/stream_message()
│   ├── prompts/
│   │   ├── concierge.txt
│   │   ├── intake.txt
│   │   └── router.txt
│   ├── services/
│   │   ├── __init__.py
│   │   └── shop_service.py     handle_message(): router -> agent
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── history.py          estimate_tokens(), trim_history()
│   │   ├── logger.py           get_logger()
│   │   └── prompts.py          load_prompt(name)
│   ├── __init__.py
│   ├── config.py               Config, load_config()
│   ├── exceptions.py           MissingConfigError
│   └── schemas.py              RepairTicket, RouteDecision
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py real API calls
│   └── unit/
│       ├── test_config.py      no API key needed
│       ├── test_history.py     no API key needed
│       └── test_schemas.py     no API key needed
├── .env                        your real key — never committed
├── .env.example                the setting names, no real values
├── .gitignore                  keeps .env, .venv/, __pycache__/ out
├── LICENSE                     lets others reuse your code (MIT)
├── main.py                     the terminal loop — the file you run
```

```
├── README.md                  the repo's front page
└── requirements.txt           the libraries, for pip install -r
```

Getting Started

Requirements: Python 3.10+ and an OpenAI API key.

```
git clone <this-repo-url>
cd RepairDesk-Device-Repair-Chat-And-Ticket-Intake-Assistant
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # then put your key in .env
```

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

Usage

```
python main.py
```

Type a question to chat with the Concierge, or describe a device problem to get a repair ticket. Type `/quit` to exit.

Running the Tests

```
# unit tests — no API key
python -m tests.unit.test_config
python -m tests.unit.test_history
python -m tests.unit.test_schemas

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_agents
```

Error Handling

Situation	What happens
`OPENAI_API_KEY` missing	Stops at startup with one clear line
Bad API key	Caught at startup and mid-session; exits cleanly
Too many requests (429)	One friendly line, keep going
Message too long	Drops the last message, asks the user to shorten it
Model answer isn't the right shape	The agent returns nothing, instead of passing on junk

License

MIT — see `LICENSE`.